Project: To-Do List API **CR ID:** CR-005 — AI-Assisted Task Breakdown

---

1. Technical Objective

Provide an analysis endpoint that sends a task's title, description, and due date to a configurable external LLM provider and returns a structured breakdown suggestion, without persisting anything and without coupling task creation to AI availability.

2. Proposed Solution

Add a dedicated `POST /api/tasks/ai-breakdown` endpoint, separate from the existing task-creation endpoint, backed by a new `AiBreakdownService` that builds a prompt from the request, calls a configurable external LLM provider over HTTP, parses a strict JSON response into DTOs, and returns it. No `Task` entity is touched anywhere in this flow.

3. Components to Modify

None of the existing CRUD components for `Task` require modification. This CR is purely additive — `TaskController`, `TaskServiceImpl`, `TaskRepository`, and `Task` / `TaskRequest` / `TaskResponse` are all untouched, directly satisfying FR-04/FR-05 by construction (there is no code path from this feature into task persistence).

Component	File	Change
`application.yml`	config	New `ai.provider.*` configuration block (base URL, model name, timeout)
Environment/secrets	outside the repo	AI provider API key, injected via environment variable, **not** committed to `application.yml`

New components:

Component	Purpose
`controller/AiBreakdownController.java`	New controller for `POST /api/tasks/ai-breakdown`
`service/AiBreakdownService.java` (+ impl)	Builds the prompt, calls the provider, parses and validates the response
`dto/AiBreakdownRequest.java`	`title`, `description`, `dueDate`
`dto/AiBreakdownResponse.java`	`shouldBreakDown`, `reason`, `suggestedTasks: List<SuggestedTask>`
`dto/SuggestedTask.java`	`title`, `description`, `dueDate`
`exception/AiProviderException.java`	Thrown on provider failure/timeout/invalid response

4. Database Changes (JPA/Hibernate)

None. No entity, table, or column is added or modified by this CR. This is a deliberate design outcome of FR-04/FR-05, not an oversight — the AI breakdown feature is stateless with respect to the database.

5. API Design

Decision: dedicated endpoint (Option B from the CR brief) is used, not an optional flag on the existing creation endpoint. See §9 for the reasoning.

```
POST /api/tasks/ai-breakdown
```

Example request

```
        {
          "title": "Develop Employee Attendance System",
          "description": "Create employee registration, fingerprint integration, attendance reports and an admin
dashboard.",
          "dueDate": "2026-12-30"
```

```
}
```

Example response — breakdown recommended

`200 OK`:

```
{
  "shouldBreakDown": true,
  "reason": "The task contains several distinct development activities.",
  "suggestedTasks": [
    {
      "title": "Analyze Requirements",
      "description": "Identify business rules and system requirements.",
      "dueDate": "2026-11-01"
    },
    {
      "title": "Design Database",
      "description": "Define entities, relationships and constraints.",
      "dueDate": "2026-11-15"
    },
    {
      "title": "Implement Backend APIs",
      "description": "Develop and validate required API endpoints.",
      "dueDate": "2026-12-05"
    }
  ]
}
```

(Dates shown are illustrative only, as noted in the project brief.)

Example response — no breakdown needed

`200 OK`:

```
{
  "shouldBreakDown": false,
  "reason": "The task is small and specific enough to track as a single item.",
  "suggestedTasks": []
}
```

Example response — provider failure

`502 Bad Gateway`:

```
{
  "timestamp": "2026-10-15T11:00:00Z",
  "status": 502,
  "error": "AI Provider Unavailable",
  "message": "The AI breakdown service is temporarily unavailable. Try again later, or create the task directly.",
  "path": "/api/tasks/ai-breakdown"
}
```

Existing endpoints are completely unaffected — `POST /api/tasks` gains no new parameters, flags, or behavior from this CR.

6. Processing Logic

1. `AiBreakdownController.analyze(request)` validates the incoming DTO (`title` required, `dueDate` must not be in the past if provided — same validation style as `TaskRequest`).
2. `AiBreakdownService` builds a prompt including the title, description, due date, and the current date (so the model can reason about available time until the deadline), and instructs the model to respond in a fixed JSON schema only.
3. The service calls the configured provider (e.g. DeepSeek) over HTTP with a bounded timeout.
4. The raw response is parsed strictly against the expected JSON schema (`shouldBreakDown`, `reason`, `suggestedTasks[]`).

    - If parsing fails (malformed JSON, missing required fields, or a shape that doesn't match) → treat as a provider failure (FR-06), not as "no breakdown needed."
5. Each suggested task's `dueDate`, if present, is sanity-checked (not before today, not after the original task's due date) **as a safeguard against unreasonable AI output** — out-of-range suggestions are dropped from the list rather than failing the whole response, and the `reason` field notes that some suggestions were adjusted/removed if this occurs.
6. The validated `AiBreakdownResponse` is returned as `200 OK`. **No repository or persistence call occurs anywhere in this flow.**
7. Separately, and entirely through the existing, unmodified flow: the frontend submits each approved suggestion as its own `POST /api/tasks` request, which goes through all of the project's existing validation (FR-07) exactly as any other task creation would — including, once implemented, CR-002's idempotency handling.

7. Error Handling

Scenario	HTTP Status	Notes
Request missing `title`	`400 Bad Request`	Standard validation, consistent with `TaskRequest`
AI provider times out	`502 Bad Gateway`	`AiProviderException`, new handler in `GlobalExceptionHandler`
AI provider returns malformed/unparsable JSON	`502 Bad Gateway`	Treated the same as a timeout — not surfaced as a client error
AI provider authentication fails (bad/expired key)	`502 Bad Gateway`	Logged server-side with detail; client sees a generic message, no credential detail leaked
Normal task creation (`POST /api/tasks`) while the AI provider is down	`201 Created`, unaffected	Explicitly required by FR-06; verified by an integration test that disables the AI provider and confirms task creation still works

8. Test Scenarios

See `TEST_CASES.md`. Summary: breakdown-recommended and no-breakdown-needed cases both return correctly shaped responses; no task row is ever created by this endpoint; provider failure/timeout is handled gracefully and does not affect `POST /api/tasks`; an out-of-range AI-suggested due date is filtered rather than breaking the response; a suggestion submitted afterward through the normal creation endpoint is validated exactly like any other task.

9. Technical Decision

Selected approach: a dedicated `POST /api/tasks/ai-breakdown` endpoint (Option B), separate from task creation, as the initial and preferred approach — consistent with the project brief's explicit statement that "the dedicated AI endpoint is the preferred initial approach."

Reason for selection:

- **Separation of concerns:** `TaskController` / `TaskServiceImpl` stay focused solely on CRUD; AI analysis logic, prompt construction, and provider integration live in their own controller/service, not interleaved with task-persistence code.
- **Reliable task creation:** `POST /api/tasks` has no dependency on the AI provider at all — an AI outage can never affect the system's core, already-tested task-creation path (directly satisfies FR-06).
- **Compatibility with existing clients:** any existing caller of `POST /api/tasks` is completely unaffected; the new capability is additive and opt-in by virtue of being a different endpoint.

Alternative considered: an optional AI flag on the existing `POST /api/tasks` endpoint (Option A) — rejected because it would couple task creation's reliability to an external AI provider's availability (violating FR-06 by construction), and would conflate "create a task" and "analyze whether a task should become several tasks" into a single endpoint with two very different failure modes and response shapes.

10. Technical Approval

- **Prepared By:** Backend Developer
- **Reviewed By:** Tech Lead
- **Approval Status:** Pending

- **Implementation Status:** Not Started